# Lab Experiment 13 : AI-Assisted Backend Logic

## Lab Overview

In this lab, students will extend the existing Task Manager backend system by integrating AI-assisted functionality into the Node.js and PostgreSQL application developed in previous labs.

Students will explore how modern backend systems interact with AI services to generate intelligent application behavior. The lab focuses on connecting backend APIs with AI-generated content while maintaining proper backend architecture, database integration, asynchronous programming practices, and critical evaluation of AI-generated outputs.

The AI functionality will build directly on top of the existing PostgreSQL database structure created in Lab 7 and the authenticated backend API system developed in Labs 8 and 9.

By the end of this lab, students will have implemented AI-powered backend features capable of generating and storing tasks dynamically within PostgreSQL.

## Learning Objectives

By completing this lab, you will learn how to:

- Integrate AI services into backend applications
- Design AI-assisted backend workflows
- Build asynchronous AI-powered API endpoints
- Store AI-generated content in PostgreSQL
- Connect Express routes to AI responses
- Critically evaluate AI-generated backend outputs
- Handle AI response validation and sanitization
- Implement structured backend logic using separation of concerns
- Build scalable AI-assisted APIs
- Understand real-world AI engineering workflows

**Technologies Used**

- Node.js: JavaScript runtime
- Express.js: Backend framework
- PostgreSQL: Relational database
- OpenAI API (or simulated AI service): AI-generated task creation
- dotenv: Environment variable management
- Postman: API testing
- pg: PostgreSQL integration

**Existing Database Structure**

This lab is built on the database created in Lab 10.

Existing tables:

**Users**

| Name | Data type |
|---|---|
| id | integer |
| email | character varying |
| password_has | text |
| created_at | timestamp without time zone |

**tasks**

| Name | Data type |
|---|---|
| id | integer |
| title | character varying |
| description | text |
| status | character varying |
| user_id | integer |
| created_at | timestamp without time zone |

## What is AI-Assisted Backend Development?

AI-assisted backend development refers to backend systems that can generate intelligent outputs, automate content creation, summarize information, prioritize tasks, analyze user requests, and interact dynamically with users. Modern applications increasingly integrate AI services into APIs to provide smarter, more personalized, and automated user experiences.

Examples include:

- ChatGPT integrations
- recommendation systems
- automated scheduling
- AI-generated content
- workflow assistants

### Real-World AI Backend Architecture

Typical AI-assisted backend workflow:
Client Request → Express API Endpoint → AI Service Request → AI Response → Backend Processing → PostgreSQL Storage → API Response to Client

### Feature Options

Students may implement one or both features.

**Option 1 — Smart Task Generator**

User submits a request such as:
Plan my week
The AI service generates suggested tasks.
Example generated tasks:

- Study algorithms
- Finish database assignment
- Attend gym session
- Schedule internship applications
- Prepare project presentation

The backend stores generated tasks inside PostgreSQL.

**Option 2 — Task Summarizer**

The backend retrieves tasks from PostgreSQL.
The AI service then:

- summarizes workload
- identifies priorities
- suggests scheduling improvements
- recommends important tasks first

Example:
You have 12 pending tasks.
High-priority tasks include project submission and exam preparation.

**Why AI Integration Matters**

Modern backend engineers increasingly build systems that:

- consume AI APIs
- process AI-generated data
- validate AI responses
- integrate intelligent workflows
- combine databases with AI outputs

This lab introduces foundational AI engineering concepts used in industry.

## Lab Procedure

### Step 1 — Install Required Packages

Open terminal.
Install:
```
npm install openai
```

**Alternative Option**
If OpenAI API access is unavailable, instructors may simulate AI responses using mock data.
Example:
```
const tasks = [
     'Study Node.js',
     'Complete database homework',
     'Prepare interview questions'
];
```

This still allows students to practice backend AI workflows.

### Step 2 — Create AI Service Folder

Create:
```
/services
```

Create:
```
aiService.js
```

**Why Use a Service Layer?**

Separating AI logic into services improves:

- maintainability
- scalability
- code organization
- testing
- separation of concerns

Professional backend systems commonly isolate external integrations inside service layers.

**Step 3 — Add OpenAI API Key**
Inside:
`.env`

Add:
```
OPENAI_API_KEY=your_api_key
```

Security Reminder
Never:

- hardcode API keys
- upload secrets to GitHub
- expose keys publicly

Always use environment variables.

**Step 4 — Configure OpenAI Client**
Inside:
`aiService.js`

Add:
```
const OpenAI = require('openai');
require('dotenv').config();

const client = new OpenAI({
    apiKey: process.env.OPENAI_API_KEY,
});

module.exports = client;
```

Understanding External APIs
The backend application is now communicating with an external AI service.
This introduces concepts including:

- external API integration
- asynchronous network requests
- service orchestration
- cloud-based AI systems

**Step 5 — Create AI Routes**
Create:
`routes/aiRoutes.js`

**Step 6 — Create Smart Task Generator Endpoint**
Inside:

```
const express = require('express');
const router = express.Router();
const db = require('../db/db');
const OpenAI = require('openai');
require('dotenv').config();

const client = new OpenAI({
    apiKey: process.env.OPENAI_API_KEY,
});

router.post('/ai/generate-tasks', async (req, res) => {
    try {
        const { prompt, user_id } = req.body;

        const aiResponse = await
        client.chat.completions.create({
            model: 'gpt-4.1-mini',
            messages: [
            {
            role: 'user',
            content: `Generate 5 tasks for: ${prompt}`,
            },
            ],
        });

        const content =
        aiResponse.choices[0].message.content;

        const taskList = content
        .split('\n')
        .filter(task => task.trim() !== '');

        const insertedTasks = [];
```

```
        for (const task of taskList) {
              result = await db.query(
                  `INSERT INTO tasks
                  (title, status, user_id)
                  VALUES ($1, $2, $3)
                  RETURNING *`,
                  [task, 'todo', user_id]
              );

              insertedTasks.push(result.rows[0]);
        }

        res.json(insertedTasks);

    } catch (error) {
        console.error(error);

        res.status(500).json({
        error: 'AI generation failed'
        });
    }
});

module.exports = router;
```

**Understanding the Workflow**

This endpoint performs multiple backend operations:

1. Receives user prompt
2. Sends prompt to AI service
3. Receives AI-generated tasks
4. Processes AI response
5. Stores tasks in PostgreSQL
6. Returns stored tasks via API

This mirrors real-world AI-assisted application workflows.

**--V**S Code showing AI route implementation

**Understanding Asynchronous AI Requests**

```
await client.chat.completions.create()
```

The server must wait for:

- internet communication
- AI processing
- response generation

This is asynchronous programming.

**Why AI Responses Must Be Validated**

AI-generated content may:

- contain incorrect formatting
- include unexpected text
- generate invalid data
- produce unsafe outputs

Backend systems must validate AI responses before storing them.

**Step 7 — Add AI Routes to Server**

Inside:

```
server.js
```

Add:

```
const aiRoutes = require('./routes/aiRoutes');

app.use('/', aiRoutes);
```

**Step 8 — Test AI Endpoint in Postman**

POST request:

```
POST http://localhost:3000/ai/generate-tasks
```

Request body:

```
{
"prompt": "Plan my week",
"user_id": 1
}
```

**Expected Result**

```
[
     {
          "id": 12,
          "title": "Finish homework"
     },
     {
          "id": 13,
          "title": "Study databases"
     }
]
```

**--p**ostman AI task generation request

--PostgreSQL tasks table showing inserted AI-generated tasks

**Step 9 — Create Task Summarizer Endpoint**

Inside:
`aiRoutes.js`

Add:

```
router.get('/ai/task-summary/:userId', async (req, res) => {
     try {
       const { userId } = req.params;

        const result = await db.query(
         'SELECT title, status FROM tasks WHERE user_id = $1',
         [userId]
        );

     const tasks = result.rows;

     const formattedTasks = tasks
     .map(task => `${task.title} (${task.status})`)
     .join('\n');
```

```
    const aiResponse = await
    client.chat.completions.create({
        model: 'gpt-4.1-mini',
        messages: [
            {
            role: 'user',
            content: `Summarize and prioritize these
            tasks:\n${formattedTasks}`,
            },
        ],
    });

    res.json({
    summary: aiResponse.choices[0].message.content,
    });

    } catch (error) {
        console.error(error);

        res.status(500).json({
        error: 'Summary generation failed'
        });
    }
});
```

**Understanding AI Summarization**

This endpoint demonstrates:

- database retrieval
- backend processing
- AI summarization
- intelligent prioritization
- API response generation

AI-enhanced backend systems commonly summarize database information.

--AI-generated task summary in Postman

## AI Prompt Engineering

Prompt quality strongly affects AI output quality.
Weak prompt:
Generate tasks
Better prompt:
Generate 5 realistic weekly study tasks for a busy computer science student.

## Why Prompt Engineering Matters

Better prompts improve:

- output accuracy
- consistency
- structure
- usefulness
- reliability

Prompt engineering is increasingly important in AI-assisted software development.

## AI Response Processing

Backend systems often need to:

- clean AI output
- parse responses
- remove formatting
- validate content
- filter unsafe responses
- transform AI data into structured formats

AI output usually requires backend processing before database storage.

## Common AI Integration Challenges

- problem: Invalid formatting, example: AI returns paragraphs instead of lists
- problem: Hallucinations, example: Incorrect or irrelevant output
- problem: Slow responses, example: External API latency
- problem: API failures, example: Rate limits or outages
- problem: Unsafe content, example: Unexpected generated responses

**Why Error Handling Matters**

External AI APIs may fail because of:

- internet connectivity
- invalid API keys
- usage limits
- malformed requests
- service outages

Always implement:
try/catch
for AI requests.

**Backend Architecture Review**

This lab now includes:

- component: PostgreSQL, responsibility: Data storage
- component: Express API, responsibility: Backend routing
- component: AI Service, responsibility: Intelligent generation
- component: Middleware, responsibility: Authentication/security
- component: Service Layer, responsibility: AI integration

This resembles modern production backend systems.

**AI-Assisted Development Activity**

Modern developers increasingly use AI tools to:

- generate backend code
- improve API design
- optimize prompts
- debug asynchronous logic
- automate workflows
- create intelligent application features

Examples:

- ChatGPT
- GitHub Copilot
- Claude

**AI Engineering Principle**
AI-generated outputs must always be:

- reviewed
- validated
- sanitized
- tested
- refined

Professional developers should never blindly trust AI-generated responses.

## Assignment Requirements

You must:

1. Generate AI-powered tasks
2. Store AI-generated tasks in PostgreSQL
3. Return generated tasks via API
4. Test AI endpoints using Postman
5. Review and improve AI-generated outputs
6. Implement proper error handling

**Bonus Challenge 1 — Task Priority Labels**
Enhance generated tasks by assigning:

- high priority
- medium priority
- low priority

Possible schema update:

```
ALTER TABLE tasks
ADD COLUMN priority VARCHAR(20);
```

**Bonus Challenge 2 — AI Productivity Recommendations**

Create endpoint:
GET /ai/productivity-tips/:userId
Possible AI output:
Focus on completing high-priority tasks before starting new tasks.

**Bonus Challenge 3 — AI Validation Layer**

Before storing tasks:

- remove duplicate tasks
- limit task length
- validate formatting
- reject unsafe content

This simulates real-world AI moderation systems.

### Common Backend Debugging Tips

Use:

```
console.log(aiResponse)
console.log(taskList)
```

Check:

1. database inserts
2. AI response formatting
3. async behavior
4. API responses
5. environment variables

## Final Deliverables

Submit:

1. Updated Node.js backend project
2. AI task generation endpoint
3. AI summarization endpoint
4. PostgreSQL integration with AI-generated data
5. Screenshots of API testing
6. AI-generated task examples
7. Error handling implementation
8. Bonus features (optional)

### Submission Checklist

- AI package installed
- AI routes created
- AI-generated tasks stored in DB
- AI summarizer implemented
- API tested in Postman

- Async/await used correctly
- Error handling added
- AI outputs validated
- Bonus challenge completed

## Conclusion

In this lab, you integrated AI-assisted functionality into a Node.js and PostgreSQL backend system by building intelligent APIs capable of generating and summarizing tasks dynamically.

You explored how modern backend systems interact with AI services, how AI-generated content is processed and stored, and how developers critically evaluate and refine AI outputs before integrating them into production systems.

This lab introduces foundational AI engineering workflows increasingly used in modern backend and cloud-native applications.